

基礎架構無痛升級戰略

從 Serverless Framework 移轉至 AWS CDK

降低維運成本，提升部署靈活性與擴充性

第一章：當前架構基礎與 Serverless Framework 的歷史價值



第二章：啟動移轉的雙重觸發點 (維護風險與強制收費)

Dual-Threat

威脅一：V.3 終止維護 (安全性風險)



Serverless Framework V.3 開源版本將於 2024 年 12 月 31 日全面停止維護 (EOL)。繼續使用將面臨無法修補的資安漏洞與技術債。

威脅二：V.4 強制收費機制 (財務衝擊)



升級至 V.4 將面臨嚴格的付費牆。年營收超過 200 萬美元的企業將被強制收取訂閱費 (Pay-As-You-Go 模式每點數 \$4 美元，或最低每月 \$142 美元起跳的預留方案，且隨服務實體數量無限遞增)。

繼續停留在 Serverless Framework 已不再是「免費且安全」的選項。我們必須在影響擴大前進行戰略性撤退。

第三章：次世代基礎架構解決方案 — AWS CDK



真實程式碼 (Infrastructure as Code)

拋棄冗長且難以除錯的 YAML 設定檔。AWS CDK 允許工程師使用 TypeScript、Python 等熟悉的程式語言來定義雲端架構，享有完整的 IDE 支援與自動完成功能。

官方支援與零授權費

AWS 官方維護的開源框架，完全免費。企業僅需為實際使用的 AWS 雲端資源付費，徹底擺脫第三方框架的訂閱綁架。

高階架構建構式 (Constructs)

內建安全且合理的預設值。以極少的程式碼即可定義複雜的架構（例如快速配置具備自動負載平衡的叢集），大幅降低人為設定錯誤的風險。

技術決策：從靜態設定邁向動態程式設計

維度	傳統宣告式 IaC (YAML/JSON)	AWS CDK (基礎設施即軟體)
心智模型	靜態設定檔	動態程式設計
表達能力	依賴特殊標籤與巨集	完全支援 (迴圈、條件式、繼承)
模組化與重用	複製貼上與繁瑣巢狀堆疊	物件導向封裝
開發體驗	缺乏即時糾錯	IDE 智能提示與編譯期檢查
品質保證	靜態 Linting	支援軟體單元測試

支柱一：打破語言藩籬的通用程式設計

無縫對接：原生支援 TypeScript, Python, Java, C#, Go。

消除壁壘：讓前端、後端與基礎設施團隊講同一種語言。

釋放表達力：完美支援條件判斷與迴圈，輕鬆應對 IoT 動態擴展需求。

單一通用語言基底
(TypeScript / Python)

邊緣設備

AWS IoT Core

Lambda 處理邏輯

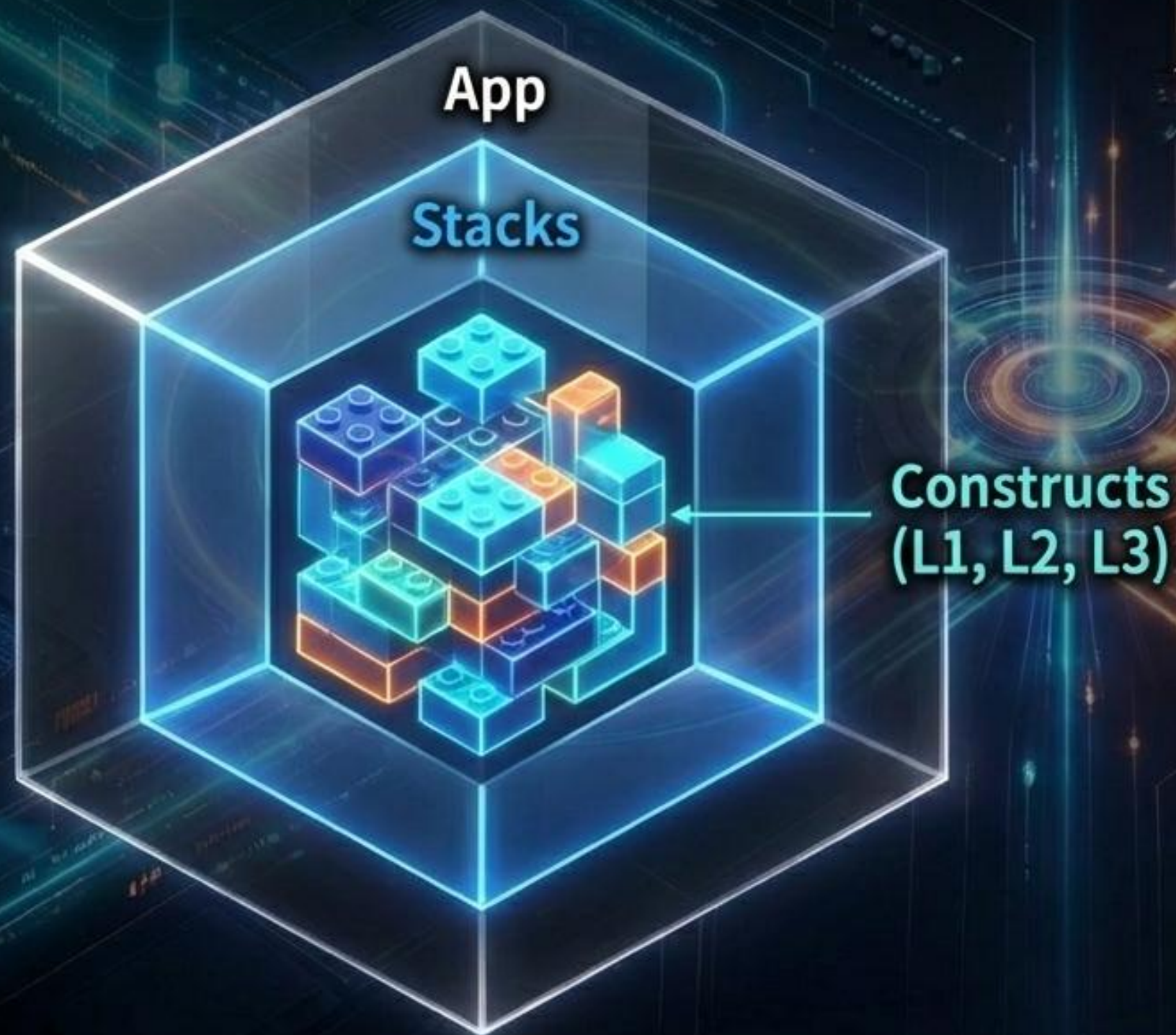
DynamoDB 儲存

支柱二：為基礎設施注入軟體工程最佳實踐



物件導向 (OOP) 建模：利用類別繼承與組合來塑造極度複雜的系統設計。

支柱三：模組化建構元件 (Constructs)



L1 建構元件 (底層)

對應單一 CloudFormation 資源，提供最精細的控制。

L2 建構元件 (高階)

帶有豐富 API 與預設安全值的 AWS 服務封裝。

L3 建構元件 (模式)

企業級架構模式，如負載平衡的 Fargate 叢集或 IoT 攝取管道。

高階抽象的爆發力：從 20 行程式碼到 50+ 資源

```
1 import * as cdk from 'aws-cdk-lib';
2 import * as ec2 from 'aws-cdk-lib/aws-ec2';
3 import * as ecs from 'aws-cdk-lib/aws-ecs';
4 import * as ecs_patterns from 'aws-cdk-lib/aws-ecs-patterns';
5 export class R2ForgetTask extends cdk.Stack {
6   constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
7     super(scope, id, props);
8
9     const vpc = new ec2.Vpc(this, 'Vpc', { maxAzs: 3 });
10
11     const cluster = new ecs.Cluster(this, 'MyCluster', { vpc });
12
13     new ecs_patterns.ApplicationLoadBalancedFargateService(this, 'MyService', {
14       cluster,
15       taskDefinitionOptions: {
16         image: ecs.ContainerImage.fromRegistry('amazon/amazon-ecs-ami'),
17       },
18     });
19   }
20 }
```

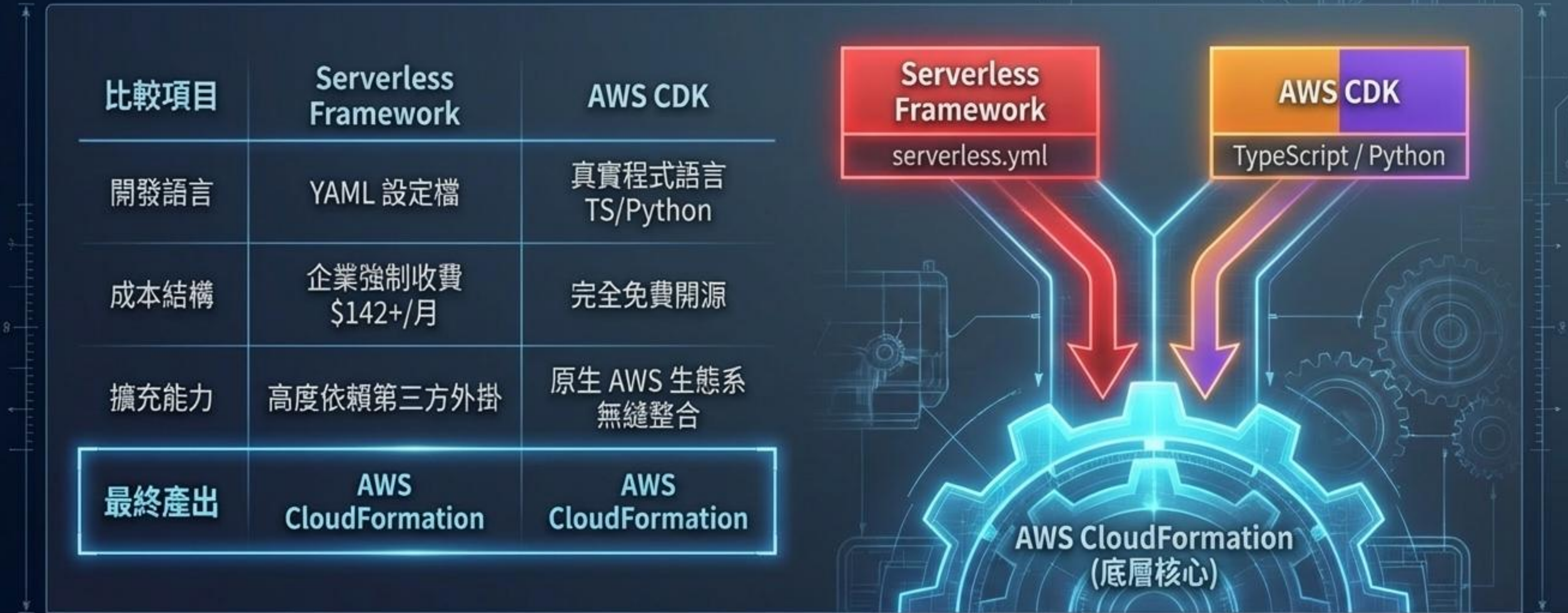
20 行高階程式碼

CDK Synthesizer



自動生成超過 500 行的
CloudFormation 藍圖

第四章：架構比較與關鍵共識（殊途同歸的底層引擎）



這是移轉戰略的核心：我們只是更換了「方向盤」，而底層運作的「引擎」（CloudFormation）始終如一。這為零停機移轉創造了完美條件。

第五章：CDK Import 移轉戰略 (實踐零停機無痛切換)



第六章：未來維運優勢 (透過 CloudFormation 實現線上熱更新)

線上配置變更 (Online Configuration)

未來所有基礎架構的變更，皆透過部署 CDK 生成的 CloudFormation 範本來執行。

無痛性能升級

需調整記憶體上限 (Memory Limit) 或更換執行環境 (Runtime) 時，可直接在程式碼中修改參數，CloudFormation 會以不中斷服務的方式進行滾動式升級 (Rolling Update)。

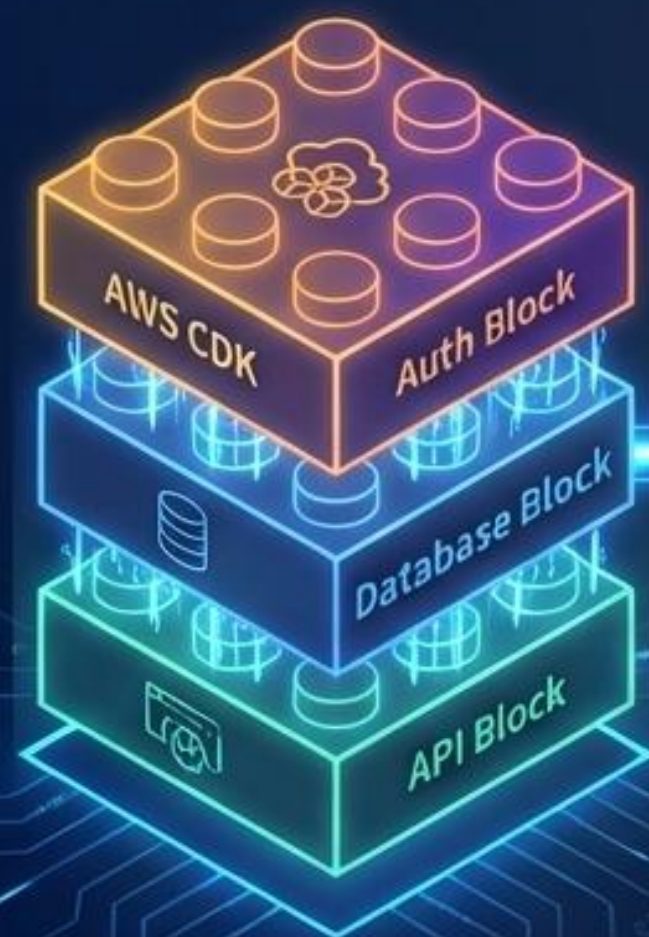
程式碼無縫抽換

部署新版本的應用程式碼時，流量將自動平滑轉移至新版本資源，確保服務不中斷。



擺脫手動進控制台 (Console) 點擊的風險，實現維運操作的全面自動化與版本控制 (GitOps)，具備極高的災難復原能力。

第七章：規模化回報 (模組化元件加速新產品建置)



標準化建構式 (Standardized Constructs)

透過真實程式語言的特性，我們能將公司專屬的資安規範與最佳實踐，封裝成可重複使用的 CDK 基礎架構模組 (如同樂高積木)。



一鍵複製環境

當業務單位需要推出新產品或建立新的測試環境時，不再需要從頭配置冗長的設定檔。



極速交付

僅需呼叫並執行一次既有的 CDK 腳本，即可在數分鐘內，精準且安全地複製出與主產品完全一致的基礎架構環境。

將基礎架構的建置時間從「數天」縮短至「數分鐘」，大幅提升研發團隊對市場需求的反應速度。

架構師的全局洞察



“AWS CDK 不是一套新的基礎設施工具。它是將基礎設施團隊轉型為高績效軟體開發團隊的催化劑。

總結與下一步行動 (Executive Summary)

[✓] 風險解除 (Risk Mitigated)

在年底 V.3 EOL 前完成移轉，徹底消除資安與維護盲區。

[✓] 成本規避 (Cost Avoided)

避開 V.4 版本針對高營收企業的強制收費與無上限的實體計費 (\$142+/月)。

[✓] 安全移轉 (Migration Safe)

憑藉底層相同 (CloudFormation) 的特性，以 CDK Import 達成零停機熱切換。

[✓] 面向未來 (Future Ready)

獲得真實程式碼的擴充能力與模組化部署優勢，加速未來產品迭代。

CloudFormation

下一步行動：請求批准

啟動 CDK Import 概念驗證 (Proof of Concept, PoC)，並針對單一非核心微服務進行零停機移轉測試。